

Disciplina: Teste de Software
Trabalho: Test Pattern
Professor: Cleiton Tavares
Aluno: Raphael Sena Augusto de Brito
Matrícula: 699651
Data: 31/05/2026

1. Padrões de Criação de Dados (Builders)

1.1 Por que CarrinhoBuilder e não CarrinhoMother?

O padrão Object Mother é indicado para entidades simples, com poucos atributos variáveis e que raramente precisam ser configuradas de formas diferentes entre os testes. O UserMother se encaixa bem nesse perfil: um usuário tem tipo fixo (PADRAO ou PREMIUM), e os testes raramente precisam de variações além dessas duas.

O Carrinho, por outro lado, é um objeto composto e variável: pode pertencer a usuários diferentes, conter zero, um ou vários itens, e o total calculado depende diretamente dessa composição. Se usássemos um CarrinhoMother, precisaríamos de um método para cada combinação relevante nos testes:

```
// Explosão de métodos — Object Mother inadequado para o Carrinho
CarrinhoMother.carrinhoVazioDeUsuarioPadrao()
CarrinhoMother.carrinhoComUmItemDeUsuarioPadrao()
CarrinhoMother.carrinhoComDoisItensDeUsuarioPremium()
CarrinhoMother.carrinhoComDoisItensDeR$200DeUsuarioPremium()
// ... e assim por diante
```

Cada novo cenário de teste exigiria um novo método, tornando a classe Mother um repositório difícil de manter. O Data Builder resolve isso ao oferecer uma API fluente que expõe explicitamente apenas o que importa para cada cenário.

1.2 Antes e Depois do Data Builder

Antes — setup manual (sem Builder):

```
it('deve aplicar desconto para cliente Premium', async () => {
  // Setup manual, verboso e frágil
  const user = new User(2, 'Maria Souza', 'premium@email.com', 'PREMIUM');
  const item1 = new Item('Produto A', 100);
  const item2 = new Item('Produto B', 100);
  const carrinho = new Carrinho(user, [item1, item2]);
  // ... continua
```

```
});
```

Neste cenário, o leitor precisa processar toda a construção manual para entender o que é relevante para o teste: é o usuário Premium que importa? São os dois itens? O total de R\$200? Nada fica explícito.

Depois — setup com Data Builder:

```
it('deve aplicar desconto para cliente Premium', async () => {
  const userPremium = UserMother.umUsuarioPremium();
  const carrinho = new CarrinhoBuilder()
    .comUser(userPremium)
    .comItens([new Item('Produto A', 100), new Item('Produto B', 100)])
    .build();
  // ... continua
});
```

Agora a intenção é imediata: este teste é sobre um usuário Premium com R\$200 em itens. Os demais detalhes do carrinho (valores padrão do construtor) ficam ocultos porque são irrelevantes para o cenário.

1.3 Como o Builder melhora legibilidade e manutenção

- Legibilidade: A API fluente funciona como uma sentença: `new CarrinhoBuilder().comUser(premium).comItens([...]).build()` lê-se quase como linguagem natural. O que é relevante para o teste fica explícito; o que é irrelevante fica nos valores padrão do construtor.
- Manutenção: Se o construtor de Carrinho ganhar um novo campo obrigatório, basta atualizar o CarrinhoBuilder. Todos os testes que o utilizam continuam compilando e passando sem alteração.
- Prevenção de Test Smells: Elimina o smell de "Setup Obscuro" — onde o leitor não consegue distinguir quais dados do Arrange são relevantes para o Assert.

2. Padrões de Test Doubles (Mocks vs. Stubs)

2.1 O teste de referência: cliente Premium com sucesso

```
describe('quando um cliente Premium finaliza a compra', () => {
  it('deve aplicar desconto de 10% e cobrar o valor correto', async () => {
    {
      // Arrange
      const userPremium = UserMother.umUsuarioPremium();
      const carrinho = new CarrinhoBuilder()
        .comUser(userPremium)
        .comItens([new Item('Produto A', 100), new Item('Produto B',
100)])
        .build();
      const cartaoCredito = { numero: '9876-5432-1098-7654' };
    }
  });
});
```

```

const pedidoSalvo = new Pedido(42, carrinho, 180, 'PROCESSADO');

const gatewayStub = { cobrar: jest.fn().mockResolvedValue({
  success: true }) };
const repositoryStub = { salvar:
  jest.fn().mockResolvedValue(pedidoSalvo) };
const emailMock = { enviarEmail:
  jest.fn().mockResolvedValue(undefined) };

const service = new CheckoutService(gatewayStub, repositoryStub,
  emailMock);

// Act
const resultado = await service.processarPedido(carrinho,
  cartaoCredito);

// Assert
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, cartaoCredito);
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com',
  'Seu Pedido foi Aprovado!',
  expect.any(String)
);
expect(resultado).toEqual(pedidoSalvo);
});
});

```

2.2 Identificação dos doubles

Dependência	Tipo de double	Papel no teste
GatewayPagamento	Stub	Controla o retorno ({ success: true }) para guiar o fluxo do SUT
PedidoRepository	Stub	Controla o retorno (pedido com id=42) para o SUT poder continuar
EmailService	Mock	Objeto sob observação — verificamos se foi chamado e com quais argumentos

2.3 Por que GatewayPagamento é um Stub?

O GatewayPagamento precisa existir no teste porque o CheckoutService chama cobrar() e toma uma decisão com base no retorno (success: true/false). O objetivo do Stub é controlar o fluxo: forçar o cenário de sucesso para que o código continue até o ponto que queremos testar.

A pergunta central do teste não é "o gateway foi chamado?", mas sim: dado que o pagamento foi aprovado, o sistema se comportou corretamente? Isso é Verificação de Estado — interessa o estado final do objeto retornado, não o comportamento do gateway em si.

Stubs são usados quando a dependência precisa retornar algo para o SUT avançar, mas não é ela o foco da verificação.

2.4 Por que EmailService é um Mock?

O EmailService não retorna nada relevante para a lógica do CheckoutService, ele é um efeito colateral. Se não verificarmos explicitamente se ele foi chamado (e com quais argumentos), o teste não tem como garantir que o e-mail de confirmação foi enviado.

A pergunta central aqui é a interação: "o sistema notificou o cliente com o e-mail correto, com o assunto correto?". Isso é Verificação de Comportamento, interessa o que foi feito, não o que foi retornado.

Mocks são usados quando a dependência é o ponto de verificação do teste: queremos garantir que ela foi acionada, quantas vezes, e com quais dados.

3. Conclusão

O uso deliberado de padrões de teste resolve problemas concretos de sustentabilidade da suíte. Os padrões de criação (Object Mother e Data Builder) atacam o Test Smell de "Setup Obscuro": quando o Arrange de um teste exige construção manual de objetos complexos, o leitor perde tempo distinguindo o que é relevante do que é ruído. O Builder torna o setup expressivo — apenas o que muda entre cenários aparece explicitamente no teste.

Os Test Doubles (Stubs e Mocks) atacam o problema de acoplamento com dependências externas. Sem eles, um teste do CheckoutService dependeria de Os padrões de criação (Object Mother e Data Builder) atacam o Test Smell de "Setup Obscuro": quando o Arrange de um teste exige construção manual de objetos complexos, o leitor perde tempo distinguindo o que é relevante do que é ruído. O Builder torna o setup expressivo — apenas o que muda entre cenários aparece explicitamente no teste.

Os Test Doubles (Stubs e Mocks) atacam o problema de acoplamento com dependências externas. Sem eles, um teste do CheckoutService dependeria de um banco de dados real, de um gateway de pagamento e de um servidor de e-mail — tornando os testes lentos, frágeis e não-determinísticos. Mais importante, a distinção entre Stub e Mock não é apenas técnica: ela documenta a intenção do teste. Um Stub diz "preciso que isso retorne X para continuar"; um Mock diz "o ponto de verificação deste teste é esta interação".

Juntos, esses padrões produzem testes que são: rápidos e isolados, legíveis, ou seja, cada teste conta uma história clara, e resilientes a mudanças internas, pois só falham quando o comportamento observável do SUT muda, não quando detalhes de implementação são refatorados.